

Page Designer Integration with Storefront Next

Page Designer is a visual, drag-and-drop content management tool for B2C Commerce that business users use to create, schedule, and publish personalized storefront pages without extensive developer intervention. It provides a low-code environment where merchants can build engaging consumer experiences, such as homepages, campaign landing pages, and product highlights, by using a library of reusable functional components. Developers build the reusable components that merchants use.

In Storefront Next, developers build Page Designer components as React components instead of ISML templates. ISML pages and templates aren't used in Storefront Next.

How It Works for Developers

Developers build the underlying building blocks that business users later manipulate. To learn more about how to build reusable pages and components in Page

Contents

[How It Works for Developers](#)

[Core Architecture](#)[Prerequisites](#)[Implement Page Designer Components](#)[Implement Page Designer Pages](#)[Generate Cartridges](#)[Deploy Cartridges](#)[Business Manager Setup](#)[Merchant Storefront Next Customization](#)[Content Slot Migration from SFRA](#)



Architecture

The Page Designer system is built on these fundamental concepts that define how content is structured and rendered.

- Pages: Top-level containers fetched via the ShopperExperience API that serve as the foundation for the storefront experience.
- Regions: Named areas within a page or component that act as placeholders where components can be nested.
- Components: Reusable UI elements, such as heroes, carousels, and grids, that contain specific functional logic and styling.
- Component Data: Asynchronous data retrieved by component-specific loaders to populate the UI elements.

Prerequisites

- MRT Admin or User Permission: To use headless Page Designer features, merchants need one of the following permissions in Account Manager: Managed Runtime Admin (`crm-org-admin`), Managed Runtime User (`crm-org-user`), or Managed Runtime Service Admin (`crm-service-admin`). If you have one of these permissions and an MRT environment pointing to your B2C Commerce site, no additional manual configuration is needed. Starting with B2C Commerce version 26.6,



Implement Page Designer Components

Page Designer components are React components that you can decorate with metadata. You can configure the metadata in Business Manager.

Component Structure

Every Page Designer component follows this structure.



```
1 src/components/my-component
2 |— index.tsx
```

Metadata Decorators

Components use three decorators to define their metadata.

Decorator	Purpose
@Component	Registers the component with the Page Designer.
@AttributeDefinition	Defines the attributes for the component.
@RegionDefinition	Defines the regions for the component.

Basic Component Example

This code example defines a Hero Banner Page Designer component. @Component



CTA link, a call-to-action link. A fallback export renders a loading skeleton. The default export is the main Hero that shows the image, title, and optional CTA. You can configure the Hero Banner in Page Designer.

```
1 // src/components/hero/
2 import { Component } from 'react'
3 import { AttributeDefinition } from '@adobe/react-spectrum'
4
5 @Component("hero", {
6   name: "Hero Banner",
7   group: "odyssey_base",
8   description: "Prominent hero banner with image, title, and optional CTA link."
9 })
10 export class HeroMetadata {
11   @AttributeDefinition({
12     name: "Title",
13     type: "string",
14     required: false,
15   })
16   title?: string;
17
18   @AttributeDefinition({
19     name: "Image",
20     type: "image",
21     required: false,
22   })
23   imageUrl?: string;
24
25   @AttributeDefinition({
26     name: "CTA Link",
27     type: "url",
28     required: false,
29   })
30   ctaLink?: string;
31 }
32
33 // Fallback component (loading skeleton)
34 export function fallbackHero() {
35   return (
36     <div className="h-hero-fallback">
37       {title} && <span>CTA Link</span>
38     </div>
39   );
40 }
41
42 // Main component (default export)
43 export default function Hero({ title, imageUrl, ctaLink }) {
44   return (
45     <section className="hero">
46       <img src={imageUrl} alt="Hero image" />
47       <h1>{title}</h1>
48       {ctaLink && <a href={ctaLink}>CTA Link</a>}
49     </section>
50   );
51 }
```

[Try for free](#)

Type	Description
<code>string</code>	Single-line text
<code>text</code>	Multi-line text
<code>markup</code>	Rich text/HTML
<code>integer</code>	Whole numbers
<code>boolean</code>	True or false toggle
<code>image</code>	Image picker
<code>url</code>	URL input
<code>product</code>	Product picker
<code>category</code>	Category picker
<code>page</code>	Page reference
<code>enum</code>	Dropdown selection
<code>cms_record</code>	CMS content reference

Component Patterns: Fallbacks and Loaders

Fallback Components

We recommend that every Page Designer component exports a `fallback` function. The fallback skeleton is shown while the component loads.





```
6     </div>
7   );
8 }
```

Best Practices:

- Keep fallbacks lightweight (no state, effects, or data fetching).
- Match the approximate dimensions of the real component.
- Use CSS animations for loading indication.

Data Loaders

Export a `loader` object for components that need dynamic data.

This code example defines a `ProductCarousel` that gets its list from a dual loader. A server loader calls `fetchProducts` using `component.data?.categoryId` and returns products. The component loads data on the server only, therefore the client loader returns null. The component receives already-resolved data and title as props—no Promise handling in the component. It renders the title and a list of products as `ProductCard`s.

```
1 // Server-side data loader
2 export const loader = {
3   server: async ({ component }) => {
4     // Fetch data on the server
5     const products = await fetchProducts(component.data?.categoryId);
6     return { products };
7   },
8   client: async ({ component }) => {
9     // Optional: Client-side loader
10    return null;
11  },
12 };
13
14 export default function ProductCarousel({ title, data }) {
15   // `data` is already resolved
16   return (
17     <div>
18       <h2>{title}</h2>
19       {data?.products.map(product => <ProductCard key={product.id}>{product}</ProductCard>)}
20     </div>
19   );
20 }
```



Key Points:

- The `data` prop is **always resolved** when your component receives it.
- The internal Component wrapper handles Promise resolution.
- Your fallback is shown while data loads.
- Multiple components load data **in parallel**.

Components with Regions

Container components can define the regions where other components can be placed.

This code example defines a Grid Layout Page Designer component that acts as a container with two regions. `@Component` registers it and `@RegionDefinition` declares Main Content with up to 6 components, and Sidebar with up to 3 components. `GridMetadata` adds an optional `Columns` attribute. A fallback export renders a simple loading grid. The default `Grid` component renders a two-column layout and uses the `Region` component to render the nested content for main and sidebar. That way, you can place other components in each region in Page Designer.

```
1 import { Component } from '@adobe/react-spectrum'
2 import { RegionDefinition } from '@adobe/react-spectrum'
3 import { Region } from '@adobe/react-spectrum'
4
5 @Component("grid", {
6   name: "Grid Layout",
7   group: "odyssey_base",
8   description: "A flexible grid layout"
9 })
10 @RegionDefinition([
11   {
12     id: "main",
13     name: "Main Content",
14     maxComponents: 6,
15   },
16   {
17     id: "sidebar",
```



```
23 @AttributeDefinition(  
24   name: "Columns",  
25   type: "integer",  
26   defaultValue: 2,  
27 })  
28 columns?: number;  
29 }  
30  
31 export function fallback(  
32   return <div className=  
33 )  
34  
35 export default function  
36   return (  
37     <div className={`gr  
38       <div className="c  
39       {/* Nested regi  
40       <Region compone  
41       </div>  
42       <div className="c  
43       <Region compone  
44       </div>  
45     </div>  
46   );  
47 }
```

Region Definition Options

These options are available for the region definition metadata.

```
1 @RegionDefinition([  
2   {  
3     id: 'content',  
4     name: 'Content  
5     maxComponents:  
6     minComponents:  
7     componentTypeIr  
8     'sfnext.her  
9     'sfnext.pro  
10  ],  
11  componentTypeEx  
12  'sfnext.gri  
13  ],  
14  },  
15  ])
```

Implement Page Designer Pages

A Page is the route-level definition that defines where Page Designer content can be placed.



regions: banner and promoContent. The route loader loads page and category data via `fetchPageWithComponentData`. The page component renders, in order: the banner region (with a skeleton fallback), a fixed `ProductGrid` for the category, and the `promoContent` region. Banner and promo slots are configurable in Page Designer. The route's category defines the product grid.

```
1 // src/routes/category.  
2 import { PageType } from '@adobe/react-spa';  
3 import { RegionDefinition } from '@adobe/react-spa';  
4 import { Region } from '@adobe/react-spa';  
5  
6 @PageType({  
7   name: "Category Page",  
8   description: "Category Page",  
9   supportedAspectTypes: ["Category", "ProductGrid"],  
10 })  
11 @RegionDefinition([  
12   {  
13     id: "banner",  
14     name: "Category Banner",  
15     maxComponents: 1,  
16   },  
17   {  
18     id: "promoContent",  
19     name: "Promotional Content",  
20     maxComponents: 3,  
21   },  
22 ])  
23 export class CategoryPage {  
24  
25   export async function loader({ params }) {  
26     const page = await fetchPage({  
27       pageId: "category",  
28       categoryId: params.categoryId,  
29     });  
30  
31     return { page };  
32   }  
33  
34   export default function render({ page }) {  
35     return (  
36       <div>  
37         {/* Page-level region */}  
38         <Region page={page} regionId="banner" />  
39  
40         {/* Main category product grid */}  
41         <ProductGrid categoryId={page.categoryId} />  
42  
43         {/* Additional promotional content */}  
44         <Region page={page} regionId="promoContent" />  
45       </div>  
46     );  
47   }  
48 }  
49  
50 export default CategoryPage;
```



Page Metadata Options

Option	Description
name	Display name of the page. This is used in the breadcrumb navigation.
description	Page description.
supportedAspectTypes	The list of supported aspect types for this page.

Supported Aspect Types

Aspect Type	Use Case	Value
pdp	Product Detail Pages	Catalog and Product
plp	Category Pages	Catalog

Region Usage in Pages

Use **page mode** for top-level regions.

```
</> | [icon]
1 <Region
2   page={loaderData.page}
3   regionId="main" // Required
4   fallbackElement={<Skeleton>}
5   errorElement={<ErrorMessage>}
6 />
```

Use **component mode** for nested regions inside container components.

```
</> | [icon]
```



Important

Don't mix `page` and `component` props. Use one or the other.

Generate Cartridges

The build process automatically generates B2C Commerce cartridge metadata from your decorated components and pages.

Automatic Generation on Build



```
1 pnpm build
```

This runs `pnpm generate:cartridge` as part of the build process and does these steps.

1. Scans `src/` for `@Component` and `@PageType` decorators.
2. Extracts metadata from decorators.
3. Generates JSON files in `cartridges/app_storefrontnext_base/cartridge/experience/`.

Manual Generation



```
1 pnpm generate:cartridge
```

Generated Output Structure



[Try for free](#)

```
6 |         | grid.json
7 |     | pages/
8 |     | | homePage.json
9 |     | | productDetailPa
10 |    | | categoryPage.js
11 |    | aspects/
12 |    | | (aspect definit
```

Generated Metadata Format

Component metadata

(hero.json):

```
</> | |
1 {
2   "name": "Hero Banner"
3   "description": "Promi
4   "group": "odyssey_bas
5   "arch_type": "headles
6   "region_definitions":
7   "attribute_definition
8   {
9     "id": "hero",
10    "name": "Hero Bar
11    "attribute_defini
12    {
13      "id": "title"
14      "name": "Titl
15      "type": "stri
16      "required": 1
17    }
18  }
19 }
20 ]
21 }
```

Page metadata

(homePage.json):

```
</> | |
1 {
2   "name": "Home Page",
3   "description": "Main
4   "arch_type": "headles
5   "region_definitions":
6   {
7     "id": "main",
8     "name": "Main Cor
9     "max_components":
10  },
11 ],
12 "supported_aspect_typ
13 "route": "/"
14 }
```



includes `arch_type: "headless"`. This flag tells Page Designer that the page or component is rendered by the headless storefront (Storefront Next) rather than by a B2C Commerce controller. When Page Designer opens a page marked as headless, it loads the Managed Runtime (MRT) storefront URL in the editor iframe instead of a classic B2C Commerce controller-based page. This behavior enables Page Designer to render and edit React components directly.

- **route** : Automatically extracted from your file-based routing structure (for example, `src/routes/category.$categoryId.tsx` → `/category/:categoryId`). When Page Designer loads a page, the `{page_route}` variable in the iframe URL is resolved using the `route` field from the page type metadata.
- **Manual additions**: If you must add metadata manually, place JSON files in the `cartridges/` directory structure.

Deploy Cartridges

Deploy the generated cartridge to your B2C Commerce instance.

Prerequisites

Create a `dw.json` file in the `storefront-next-dev` package directory.





```
6 }
```

Deploy the Cartridge with the push Command

To deploy the cartridge, run this command.

```
1 pnpm push
```

The command:

1. Generates cartridge metadata if `GENERATE_AND_DEPLOY_CARTRIDGE_ON_MRT_PUSH` is enabled.
2. Deploys the cartridge to B2C Commerce via WebDAV.
3. Pushes the storefront bundle to Managed Runtime.

Manual Cartridge Deployment

To deploy only the cartridge without pushing to Managed Runtime (MRT), use these commands.

```
1 # Generate metadata
2 pnpm generate:cartridge
3
4 # Deploy to B2C Commerce
5 sfnext deploy-cartridge
```

Deployment Process

1. **ZIP Creation:** Cartridge directory is zipped.
2. **Upload:** ZIP uploaded to B2C Commerce via WebDAV.
3. **Unzip:** Archive extracted on the server.



Business Manager Setup

Prerequisites

1. **Cartridge Path:** Add `app_storefrontnext_base` to your site's cartridge path in Business Manager.
 - Go to **Administration > Sites > Manage Sites > [Your Site] > Settings**.
 - Add `app_storefrontnext_base` to the cartridge path.
2. **MRT Linking:** Make sure that your MRT environment is linked to your B2C Commerce instance. The MRT environment is linked if the storefront was set up in Business Manager, or you can link it in MRT's Environment Settings page. The MRT storefront URL in Page Designer is automatically configured when exactly one MRT environment is linked to the specific B2C Commerce instance and site ID. No manual URL configuration is required. Page Designer uses this URL to populate the `headless_url` object, (containing `edit_url`, `preview_url`, and `info_url`) that controls where the editor iframe points.

If the Page Designer iframe appears blank, verify that your MRT environment is correctly linked to your B2C Commerce instance and site.



Content > Page Designer.

2. Create a page or edit an existing one.
3. Select a page type that matches your Storefront Next page types.
4. Add components to regions using the visual editor.
5. Configure component attributes in the properties panel.
6. Publish the page.

Page Assignment

You can assign pages to:

- **Specific entities** - A particular product, category, or content asset
- **Default pages** - Fallback for all entities of a type

Live Preview

When editing in Page Designer:

1. Business Manager reads the `route` from page metadata.
2. Loads your storefront in an iframe with the appropriate URL.
3. Shows live preview as you configure components.

Shopper Context Preview

To preview a page in the context of a specific Customer Group or on a specific date:

1. **Enable the feature toggle in Business Manager:** Turn on the “Shopper Context” feature toggle in Business Manager.
2. **Configure MRT environment variable:** Set `PUBLIC__app__features__shopperContext__e`



Known Limitations

- **No live editing without saving:** Changes made in the Page Designer properties panel aren't reflected in the preview iframe in real time. You must save changes to see them in the preview.
- **Hybrid storefronts – link builder restriction:** In hybrid deployments (SFRA and Storefront Next running together), the Page Designer link builder can only link to pages within the same technology stack. Headless pages can link to other headless pages, and SFRA pages can link to other SFRA pages, but cross-stack linking isn't supported.

Local Development

Local development with Page Designer using `localhost` URLs isn't supported. To test Page Designer integration, deploy your storefront to an MRT environment.

As a temporary workaround, use a local development cartridge to redirect Page Designer to your local server. Download the cartridge from [Local Dev Cartridge \(ZIP\)](#) .

Setup Steps

1. Deploy the cartridge to your B2C Commerce instance.
2. Add the cartridge to your Business Manager cartridge path.
3. In Business Manager, go to **Administration > SF Next Local Dev**.



5. Run the service worker.

6. Click **Enable Headless URL Override**.

Important

Click the checkbox only once. Due to a known UI limitation, the checkbox doesn't reflect the correct state after toggling.

Troubleshooting

Issue	Solution
Components don't appear	Verify th in the ca deploye
Page doesn't load	Check th page me your rou
Attributes aren't configurable	Make su @Attri decoratc
Region is empty	Verify th matches and met
PD iframe is blank	Verify th environr the corra Commer site ID. See Runtime
Preview flickering/refreshing	Ensure t enabled configur environr Manage Adminis

Merchant Storefront Next Customization